

3.1 / 3.2 / 3.3

Transport-Layer Services, Multiplexing & UDP

Kurose & Ross, 8th Ed.



Chapter 3

Section 3.1–3.3: Introduction

The transport layer provides logical communication between application PROCESSES on different hosts

3.1 Introduction & Transport-Layer Services

What does the transport layer do?

How does it relate to the network layer (IP)?

What do UDP and TCP provide — and NOT provide?

3.2 Multiplexing and Demultiplexing

How does a host with many apps running simultaneously deliver each incoming packet to the correct application?

Port numbers, 2-tuple vs 4-tuple.

3.3 Connectionless Transport: UDP

The simplest transport protocol. Why use it?

UDP segment structure (8-byte header).

UDP checksum and the end-to-end argument.

Logical Communication Between Processes

Section 3.1.1: Introduction

From the application's perspective: the hosts appear directly connected — even if they are on opposite sides of the planet

Network Layer (IP): Host-to-Host

IP provides logical communication between entire HOSTS (machines).

IP is a best-effort delivery service:

- No guaranteed delivery
- No guaranteed order
- No guaranteed integrity

IP is UNRELIABLE.

extends →

Transport Layer: Process-to-Process

The transport layer EXTENDS host-to-host delivery into PROCESS-TO-PROCESS delivery.

A host runs many applications simultaneously — the transport layer delivers each incoming packet to the correct one.

Implemented in END SYSTEMS only — not in routers.

Application

Transport ◀

Network (IP)

Link

Physical

Transport-layer protocols run in END SYSTEMS — never in routers or switches

Two Transport Protocols: UDP and TCP

Section 3.1.2: Overview

UDP — User Datagram Protocol (RFC 768)

Unreliable data transfer

Connectionless (no handshake)

Process-to-process delivery (via port numbers)

Error detection (checksum only)

No congestion control

No flow control

TCP — Transmission Control Protocol (RFC 793)

Reliable data transfer

Connection-oriented (3-way handshake)

Process-to-process delivery

Error detection (checksum)

Congestion control

Flow control

Both protocols built on top of IP — which is unreliable. TCP converts unreliable IP into reliable process-to-process delivery. UDP does not.

The Multiplexing / Demultiplexing Problem

Section 3.2: Multiplexing and Demultiplexing

A host running many applications simultaneously — how does each incoming packet reach the right one?

Example: you have open simultaneously — 2 Telnet sessions + 1 FTP session + 1 HTTP session = 4 concurrent processes.

DEMULPLEXING (at the receiver)

Delivering the data in an arriving transport-layer segment to the **CORRECT SOCKET**.

The transport layer examines header fields in the arriving segment to identify which socket should receive it, then directs the data to that socket.

MULTIPLEXING (at the sender)

Gathering data chunks from multiple sockets, encapsulating each with header information that will later be used for demultiplexing, and passing the resulting segments to the network layer.

Analogy:

Dad (Bill) performs demultiplexing — reads each envelope and hand-delivers it to the right child.

Mom (Ann) performs multiplexing — collects outgoing letters from all children and gives the stack to the mail carrier.

Port Numbers — The Key Mechanism

Section 3.2: Multiplexing and Demultiplexing

Each socket has a unique identifier — the **PORT NUMBER** — a 16-bit integer (0–65535)

Transport-layer segment header includes:

Source
Port #

Destination
Port #

Other header fields

Application data

Port Number Ranges:

0 – 1023

Well-Known Port Numbers

Reserved for well-known protocols. Defined in RFC 1700 / IANA.

1024 – 65535

General Application Use

Assigned automatically (client ephemeral) or explicitly (server).

80

HTTP

443

HTTPS

25

SMTP

53

DNS

21

FTP

22

SSH

Connectionless Demultiplexing (UDP)

Section 3.2: Multiplexing and Demultiplexing

**A UDP socket is fully identified by a 2-TUPLE:
(destination IP address, destination port number)**

Example: Host A (port 19157) → Host B (port 46428)

1

Build segment

Transport layer at Host A creates a UDP segment:
Source port: 19157 · Destination port: 46428 · data

2

Network layer

Segment encapsulated in IP datagram and routed (best-effort) to Host B.

3

Demultiplex

Transport layer at Host B reads the destination port (46428) and delivers the segment to the socket bound to port 46428.

Key consequence of 2-tuple:

Two UDP segments with DIFFERENT source IPs or source ports — but the SAME destination IP and destination port — will BOTH be delivered to the SAME socket.

→ DNS server (port 53) receives queries from all clients through ONE socket.

Connection-Oriented Demultiplexing (TCP)

Section 3.2: Multiplexing and Demultiplexing

**A TCP socket is identified by a 4-TUPLE:
(source IP, source port, destination IP, destination port)**

Why 4-tuple instead of 2-tuple?

Two TCP segments with DIFFERENT source IP addresses or different source port numbers are directed to TWO DIFFERENT SOCKETS — even if they have the same destination IP and port.

This allows the server to maintain a SEPARATE connection with each client, each handled by its own dedicated socket.

Web Server Example — port 80:

Client A

src: 192.168.1.1 : 54321

→ DIFFERENT socket (despite same dest port 80)

Client B

src: 10.0.0.5 : 64000

→ DIFFERENT socket (despite same dest port 80)

	Socket identifier	Multiple clients → same port?
UDP	(dest IP, dest port) — 2-tuple	→ SAME socket
TCP	(src IP, src port, dest IP, dest port) — 4-tuple	→ DIFFERENT sockets

Why Choose UDP over TCP? — Four Reasons

Section 3.3: Connectionless Transport: UDP

TCP is reliable. UDP is not. So why would anyone choose UDP?

1 Fine-grained Control — What is Sent and When

UDP sends immediately — no congestion-control throttling, no retransmission delays.

Real-time apps (voice calls, video conferencing, live gaming) need data NOW.

They can tolerate occasional packet loss but cannot tolerate TCP delays.

2 No Connection Establishment Overhead

TCP requires a 3-way handshake — at least 1 full RTT of delay before any data flows.

UDP just sends. No setup, no delay.

DNS uses UDP precisely for this: a quick query/response does not justify TCP overhead.

3 No Connection State

TCP maintains state per connection: buffers, sequence numbers, timers, congestion window.

UDP maintains no per-client state → a server can support far more simultaneous clients.

4 Small Packet Header Overhead

UDP header: 8 bytes. TCP header: 20+ bytes.

For small messages (DNS queries ~50 bytes, SNMP packets), this overhead difference is significant.

UDP offers: finer control · no setup delay · less state · smaller headers — at the cost of no reliability guarantee

Applications That Use UDP

Section 3.3: Connectionless Transport: UDP

UDP is chosen whenever low latency or high control matters more than guaranteed delivery

Application	Protocol	Why UDP?
DNS	RFC 1035	Quick query/response — TCP handshake delay is unacceptable for every DNS lookup.
SNMP (Network Mgmt)	RFC 1157	Must work when network is stressed — precisely when TCP congestion control fails most.
Internet Telephony	SIP / RTP	Loss-tolerant; reacts very poorly to TCP congestion-control delays. Needs real-time delivery.
Video Conferencing	RTP / RTCP	Same as telephony. Small losses cause glitches; TCP delays cause freezing — much worse.
Online Games	Proprietary	Low latency is critical. Occasional lost game state updates are tolerable.
Streaming Media (live)	DASH / RTP	Some loss is tolerable; TCP retransmissions would cause playback pauses.
HTTP/3 / QUIC	RFC 9000	QUIC implements its own reliability + congestion control on top of UDP at the app layer.

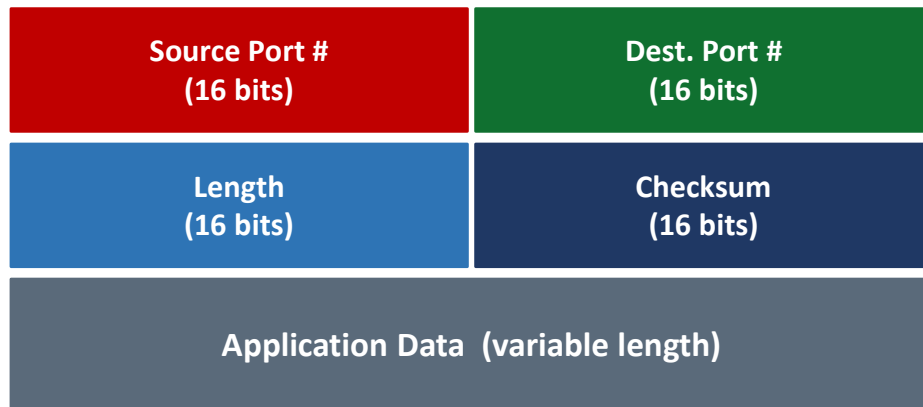
Note: organizations may block UDP for security — in practice TCP is increasingly used as fallback even for multimedia

UDP Segment Structure (RFC 768)

Section 3.3.1: UDP Segment Structure

UDP header: 8 bytes = 4 fields × 2 bytes each + variable-length application data

Segment format (32 bits = 4 bytes wide):



← 8 bytes total header →

Field descriptions:

Source Port

Port of the sending process. Serves as a return address — the receiver uses it to send replies back.

Dest. Port

Port of the receiving process. Used by transport layer to demultiplex the segment to the correct socket.

Length

Total bytes in the UDP segment (header + data). Needed because data field size varies per segment.

Checksum

1's complement error detection. Receiver checks for corruption introduced during transit.

8-byte UDP header vs 20+ byte TCP header — for a 50-byte DNS query: UDP overhead 14% vs TCP overhead 29%

UDP Checksum — Error Detection

Section 3.3.2: UDP Checksum

The UDP checksum detects whether bits in the segment have been altered during transit

Sender: Computing the Checksum

1. Treat the entire segment as a sequence of 16-bit integers.
2. Compute the 1's complement of their sum.
(Overflow wraps around — added back to result.)
3. Place the result in the Checksum field of the header.

Receiver: Checking the Checksum

1. Add all 16-bit words in the received segment INCLUDING the checksum.
2. If NO errors: sum = 1111 1111 1111 1111 (all ones)
3. If any bit is 0: an error has been introduced.

Why does UDP include a checksum if Ethernet already does error checking?

- Not all links check for errors — a segment may traverse a link with no error detection.
- Bit errors can be introduced in router MEMORY — not protected by link-layer checksums.
- End-to-end error detection at the transport layer is the only reliable guarantee.

End-to-End Argument: certain functions (like error checking) must be implemented at end systems to be correct and complete.

UDP Checksum — Worked Example

Section 3.3.2: UDP Checksum

Three 16-bit data words to sum:

Word 1: 0110 0110 0110 0000

Word 2: 0101 0101 0101 0101

Word 3: 1000 1111 0000 1100

Step 1: Add words 1+2 = 1011 1011 1011 0101

no overflow

Step 2: Add word 3 = 0100 1010 1100 0010

overflow → wrap around added back

Step 3: 1's complement (flip all bits) = 1011 0101 0011 1101

This is the CHECKSUM

Receiver adds all 4 words (data × 3 + checksum): = 1111 1111 1111 1111 → ALL ONES = no error detected ✓

If any bit is 0 in the final sum → error has been introduced.

Reliability on top of UDP & Congestion Warning

Section 3.3: Connectionless Transport: UDP

Can UDP be Made Reliable?

YES — reliability can be built into the APPLICATION on top of UDP.

The application implements:

- Acknowledgments
- Retransmission timers
- Sequence numbers

This gives the application BOTH:

- ✓ Low latency (no TCP congestion control throttling)
- ✓ Reliable delivery

Best example: QUIC (RFC 9000)

- Used by HTTP/3
- Implements reliability + congestion control + encryption at the application layer on top of UDP
- Can be updated much faster than TCP (which requires OS updates)

⚠ UDP and Congestion — A Warning

UDP has NO congestion control.

A UDP sender can pump data at any rate, as long as it wants.

This freedom has serious consequences:

If all apps used UDP at full speed:

- Routers would overflow
- Nearly all packets would be lost
- TCP senders would slow down drastically
- Internet could effectively collapse

Running high-bit-rate multimedia over UDP must be done carefully.

Many organizations block UDP traffic for security reasons → TCP is increasingly used as fallback even for streaming media.

UDP vs. TCP — Full Comparison

Section 3.1–3.3: Summary

Service / Feature	UDP	TCP
Transport type	Connectionless	Connection-oriented
Reliability	✗ Unreliable	✓ Reliable (retransmission)
Ordered delivery	✗ Not guaranteed	✓ Guaranteed (sequence numbers)
Congestion control	✗ None	✓ Yes (throttles sender)
Flow control	✗ None	✓ Yes (receive window)
Connection setup	None — send immediately	3-way handshake (≥ 1 RTT delay)
Header size	8 bytes	20+ bytes
Socket identifier	2-tuple (dest IP, dest port)	4-tuple (src IP, src port, dest IP, dest port)
Mux/demux	✓ Yes	✓ Yes
Error detection	✓ Checksum (weak)	✓ Checksum
Encryption (built-in)	✗ No (use DTLS)	✗ No (use TLS)
Typical applications	DNS, SNMP, games, QUIC	Web, email, file transfer, SSH

Key Definitions — Sections 3.1–3.3

Summary

Logical communication	From the app's view, hosts appear directly connected — regardless of actual network topology.
Segment	The name for a transport-layer packet. Both UDP and TCP packets are called segments in this text.
Multiplexing	Gathering data from multiple source sockets, adding header fields, passing to the network layer.
Demultiplexing	Delivering data in arriving segments to the correct socket using the destination port number.
Port number	16-bit integer (0–65535) identifying a socket. Well-known ports 0–1023 reserved by IANA.
UDP 2-tuple	(dest IP, dest port) — fully identifies a UDP socket. Source port acts as return address.
TCP 4-tuple	(src IP, src port, dest IP, dest port) — each TCP connection gets its own socket.
UDP checksum	1's complement of sum of all 16-bit words. Detects errors; cannot correct them.
End-to-end argument	Error checking must be done at end systems — not in the network — to be complete and correct.
QUIC	Application-layer protocol providing reliability + congestion control + encryption on top of UDP.

Summary

3.1 / 3.2 / 3.3 — Transport Layer Services, Multiplexing & UDP

- Transport layer provides LOGICAL COMMUNICATION between PROCESSES — extends IP's host-to-host delivery to process-to-process.
- IP (network layer) is unreliable — best effort only. TCP converts it to reliable delivery. UDP does not.
- MULTIPLEXING: gather data from sockets + add port numbers + send. DEMULTIPLEXING: use dest port to route to correct socket.
- Port numbers: 0–65535. Well-known ports 0–1023 (HTTP=80, DNS=53, SMTP=25). Managed by IANA.
- UDP demultiplexing: 2-tuple (dest IP, dest port). Multiple clients → SAME UDP socket.
- TCP demultiplexing: 4-tuple (src IP, src port, dest IP, dest port). Each connection → DIFFERENT socket.
- UDP: connectionless, unreliable, 8-byte header. Chosen for: speed, no setup delay, less state, small overhead.
- UDP segment: Source Port + Dest Port + Length + Checksum (8 bytes) + application data.
- UDP checksum: 1's complement sum of 16-bit words. Receiver checks: all-ones = no error. End-to-end argument.
- Reliability CAN be built on top of UDP (QUIC). UDP has no congestion control — use responsibly.